field, the `value_count` aggregation does what we need. For example, running the following query returns the number of values in our stash for the `best_seller` field.

Listing 13.2 Finding the number of values for a field

```
GET tv_sales/_search
{
  size = 0
  "aggs": {
    "total-number-of-values": {
      "value_count": {
        "field": "best_seller"
      }
    }
  }
}
```

Names the aggregation results

Names the aggregation (value_count)

Field on which the value_count is performed

The `value_count` aggregation is carried out on the `best_seller` field. Note that by default, aggregations are not executed on `text` fields. In our sample data, the `best_seller` field is a `boolean` data type and a good candidate for the `value_count` metric aggregation. Running the query in listing 13.2 outputs the following:

```
"aggregations" : {
    "total-values" : {
      "value" : 2
    }
  }
```

There are two values (two documents) for the `best_seller` field. Note that `value_count` doesn't pick unique values; it does not remove duplicate values for the specified field across the document set.

Aggregations on text fields are not optimized

Text fields do not support sorting, scripting, and aggregations. Aggregations are ideally carried out on nontext fields such as `number`, `keyword`, `boolean`, and so on. Because `text` fields are not optimized for aggregations, by default, Elasticsearch stops us from creating aggregation queries on them. If you are curious, try an aggregation on a `text` field like the name field, and see what exception Elasticsearch throws:

```
"root_cause" : [
{
  "type" : "illegal_argument_exception",
  "reason" : "Text fields are not optimised for operations that require
  ➥ per-document field data like aggregations and sorting, so these
  ➥ operations are disabled by default. Please use a keyword field
  ➥ instead. Alternatively, set fielddata=true on [name] in order to
  ➥ load field data by uninverting the inverted index. Note that this
  ➥ can use significant memory."
 }
```